

Hydraulic Formulas Used

simulator.py — Telemetry Generation

```
# NORMAL mode: water flow
water_flow = min(1.5, round(0.15 + (occupancy / 25) * random.uniform(0.7, 1.35), 2))

# NORMAL mode: flush count
flush_count = min(occupancy, max(0, int(random.gauss(occupancy * 0.35, 2))))
```

analytics.py — Core Analytics Engine

```
# Rolling Z-Score
z_score = (value - mean(history)) / stdev(history)

# Z-Score anomaly flag
z_anomaly = abs(z_score) >= 2.5 # Z_SCORE_THRESHOLD

# Leak detection condition (per tick)
is_leak_tick = (water_flow > 0.25) and (occupancy == 0)

# Leak confirmed after streak
continuous_leak = leak_streak > 5 # LEAK_TICKS_REQUIRED

# Dynamic Hygiene Index
hygiene_score = round(max(0.0, min(100.0, 100 - (0.6 * occupancy) + (0.8 * flush_count))), 2)

# Hygiene breach flag
hygiene_breach = hygiene_score < 30 # HYGIENE_BREACH_THRESHOLD
```

page.tsx — Frontend Health Calculations

```
// Average Hygiene Score across all fixtures
averageHygiene = sum(fixture.hygiene_score for all fixtures) / total_fixtures

// Fleet Availability
availability = ((normal_count + hygiene_warning_count) / total_fixtures) * 100

// Overall Health Score
```

```
healthScore = clamp(round(averageHygiene * 0.6 + availability * 0.4 - leaks * 8), 0, 100)
```

```
// Hygiene Compliance (displayed as %)
```

```
hygieneCompliance = round(averageHygiene)
```

```
// Water Efficiency
```

```
waterEfficiency = clamp(100 - leaks * 3, 70, 100)
```

```
// Fleet Availability (displayed as %)
```

```
fleetAvailability = round(((total_fixtures - leak_count) / total_fixtures) * 1000) / 10
```

main.py — KPIs & Severity

```
# Severity score assigned per status
```

```
severity = 0.95 # if status == LEAK
```

```
severity = 0.75 # if status == HYGIENE_WARNING
```

```
severity = 0.10 # if status == NORMAL
```

```
# System uptime
```

```
uptime = round(((total_fixtures - active_leaks) / total_fixtures) * 100, 1)
```

```
# Total water saved
```

```
total_water_saved = base_water_saved_liters + water_saved_accumulated # base = 1248.0 L
```

```
# Cost saved
```

```
total_cost_saved = round(total_water_saved * 0.11, 2) # ₹0.11 per liter
```

```
# Water saved on ticket completion
```

```
water_saved_accumulated += work_order["water_saved_lpm"]
```

agent.py — Work Order Water Savings

```
# Water saved estimate (fallback when agent unavailable)
```

```
water_saved_lpm = round(fixture["water_flow_lpm"] * 60, 1)
```

```
# From Gemini output (when agent works)
```

```
estimated_water_saved_lph = anomalous_flow_lph - normal_baseline_lph
```

```
# Baselines:
```

```
# Commercial lavatory: 1.9 lph
```

```
# Sensor faucet idle leak: 0.5 lph
```